

ĐẠI HỌC BÁCH KHOA HÀ NỘI

BÁO CÁO BÀI TẬP LỚN

Nhóm 8 - Chủ đề 12

**Các phương pháp tối ưu cho bài toán người du lịch
với ràng buộc cửa sổ thời gian**

VŨ TIẾN ĐẠT

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: TS. Nguyễn Văn Sơn

Khoa: Khoa học Máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN.....	1
CHƯƠNG 2. MÔ TẢ BÀI TOÁN.....	2
CHƯƠNG 3. CÁC PHƯƠNG PHÁP GIẢI CHÍNH XÁC	3
3.1 Phương pháp quay lui.....	3
3.2 Phương pháp quy hoạch động với mặt nạ bit.....	4
CHƯƠNG 4. MÔ HÌNH HÓA BÀI TOÁN.....	7
4.1 Mô hình quy hoạch nguyên tuyến tính cho bộ giải PywrapLP	7
4.2 Mô hình quy hoạch ràng buộc cho bộ giải CP-SAT	8
CHƯƠNG 5. CÁC PHƯƠNG PHÁP THAM LAM	9
CHƯƠNG 6. CÁC PHƯƠNG PHÁP TÌM KIẾM CỤC BỘ	10
6.1 Tổng quan về các phương pháp tìm kiếm cục bộ được sử dụng	10
6.1.1 Khởi tạo lời giải ban đầu.....	10
6.1.2 Toán tử biến đổi	10
6.1.3 Đánh giá chất lượng lời giải	11
6.2 Thuật toán tìm kiếm leo đồi	11
6.3 Thuật toán mô phỏng quá trình tôi ủ vật liệu	12
CHƯƠNG 7. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	14
7.1 Cấu hình thực nghiệm	14
7.1.1 Bộ dữ liệu thử nghiệm	14
7.1.2 Cài đặt tham số	15
7.2 Kết quả của các bộ giải chính xác thuộc thư viện OR-Tools.....	15
7.3 Kết quả của các phương pháp tìm kiếm cục bộ	16
7.3.1 Thuật toán tìm kiếm leo đồi	16
7.3.2 Thuật toán tìm kiếm mô phỏng quá trình tôi ủ vật liệu	17

7.4 Phân tích mức độ đóng góp của các toán tử	17
7.5 Thống kê về các lần cập nhật lời giải của thuật toán tìm kiếm mô phỏng quá trình tối ưu vật liệu.....	18
CHƯƠNG 8. KẾT LUẬN	20
TÀI LIỆU THAM KHẢO.....	21

DANH MỤC HÌNH VẼ

Hình 7.1	Số lần cải thiện lời giải của từng toán tử trong thuật toán tìm kiếm leo đồi.	17
Hình 7.2	Số lần cập nhật lời giải của từng toán tử trong thuật toán tìm kiếm mô phỏng quá trình tôi ủ vật liệu, bao gồm các lần cải thiện lời giải và các lần chấp nhận lời giải tệ hơn để thoát khỏi điểm tối ưu cục bộ.	18
Hình 7.3	Thống kê về các lần cập nhật lời giải của từng toán tử trong thuật toán tìm kiếm mô phỏng quá trình tôi ủ vật liệu.	19

DANH MỤC BẢNG BIỂU

- Bảng 7.1 Kết quả thực nghiệm của các bộ giải chính xác. Thời gian thực thi được làm tròn đến ba chữ số sau dấu thập phân. Giá trị được in đậm là giá trị tối ưu nếu bộ giải tìm được lời giải tối ưu. Giá trị hàm mục tiêu và thời gian thực thi là $+\infty$ nếu bộ giải không tìm được một lời giải chấp nhận được trong thời gian cho phép. . . . 14
- Bảng 7.2 Kết quả thực nghiệm trên ba lần chạy độc lập của thuật toán tìm kiếm leo đồi. Giá trị trung bình và độ lệch chuẩn của các giá trị hàm mục tiêu được làm tròn đến số nguyên gần nhất. Số vòng lặp là tổng số vòng lặp của hai cách khởi tạo lời giải. 15
- Bảng 7.3 Kết quả thực nghiệm trên ba lần chạy độc lập của thuật toán tìm kiếm mô phỏng quá trình tôi ủ vật liệu. Giá trị trung bình và độ lệch chuẩn của các giá trị hàm mục tiêu được làm tròn đến số nguyên gần nhất. 16

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN

Bài toán người du lịch với ràng buộc cửa sổ thời gian (Travelling Salesman Problem with Time Windows - TSPTW) là một biến thể của bài toán người du lịch cổ điển (Travelling Salesman Problem - TSP) và cũng là một trường hợp đặc biệt của bài toán định tuyến phương tiện với ràng buộc cửa sổ thời gian (Vehicle Routing Problem with Time Windows - VRPTW) khi chỉ có một phương tiện duy nhất [1]. Trong bài toán này, một người giao hàng sẽ xuất phát từ kho hàng, đi qua một tập các khách hàng, mỗi khách hàng đúng một lần, và quay lại kho hàng ban đầu. Khác với bài toán cổ điển, thời điểm phục vụ mỗi khách hàng trong TSPTW phải nằm trong một khoảng thời gian cho trước, nên lời giải không chỉ cần tối ưu tổng thời gian di chuyển mà còn phải bảo đảm các ràng buộc về thời điểm phục vụ từng khách hàng.

TSPTW có ý nghĩa thực tiễn trong nhiều khía cạnh của đời sống hiện đại. Trong lĩnh vực logistics, bài toán xuất hiện khi doanh nghiệp cần lập lộ trình giao hàng cho khách hàng với khung giờ nhận hàng cụ thể [2]. Trong vận tải hành khách, TSPTW có thể giúp mô hình hóa việc đón trả khách theo lịch hẹn [3]. Đối với các dịch vụ giao hàng, bài toán hướng đến việc giúp nhân viên di chuyển hiệu quả nhưng vẫn đến đúng thời gian đã cam kết [4]. Việc giải quyết tốt TSPTW giúp giảm chi phí vận hành, tiết kiệm thời gian, nâng cao chất lượng dịch vụ và sử dụng hiệu quả nguồn lực.

Tuy nhiên, TSPTW là một bài toán tối ưu tổ hợp có độ phức tạp cao. Khi số lượng khách hàng cần phục vụ tăng lên, số lượng phương án lộ trình có thể xét tăng rất nhanh theo hàm mũ, trong khi các ràng buộc về cửa sổ thời gian làm cho việc tìm lời giải khả thi trở nên khó khăn hơn. Do đó, việc nghiên cứu, cài đặt và đánh giá các phương pháp giải TSPTW là cần thiết để lựa chọn cách tiếp cận phù hợp với từng quy mô và đặc điểm dữ liệu khác nhau.

Báo cáo này hướng tới việc trình bày, thử nghiệm và đánh giá các phương pháp giải quyết bài toán TSPTW. Báo cáo mô tả cách mô hình hóa bài toán, các ràng buộc cần thỏa mãn, tiêu chí đánh giá chất lượng lời giải và quy trình thực nghiệm đối với từng phương pháp được lựa chọn. Thông qua việc thử nghiệm trên các bộ dữ liệu khác nhau, báo cáo chỉ ra điểm mạnh và điểm yếu của từng phương pháp. Các kết quả thực nghiệm được xem xét dựa trên những tiêu chí như chất lượng lời giải, thời gian chạy và mức độ ổn định khi kích thước bài toán thay đổi. Toàn bộ mã nguồn cài đặt và dữ liệu thực nghiệm của báo cáo đều đã được công khai¹.

¹<https://github.com/datvt029999/tsptw>

CHƯƠNG 2. MÔ TẢ BÀI TOÁN

Trong TSPTW, một nhân viên giao hàng cần di chuyển từ kho hàng để phục vụ một tập gồm n khách hàng ($n \leq 1000$). Kho hàng được ký hiệu là điểm 0 còn n khách hàng được ký hiệu lần lượt là các điểm $1, 2, \dots, n$. Nhân viên giao hàng xuất phát từ kho tại thời điểm $t_0 = 0$, sau đó phải đi qua mỗi khách hàng đúng một lần để hoàn thành việc giao hàng.

Mỗi khách hàng i có một cửa sổ thời gian phục vụ $[e_i, l_i]$. e_i là thời điểm sớm nhất mà nhân viên có thể bắt đầu giao hàng cho khách hàng i còn l_i là thời điểm muộn nhất mà việc phục vụ khách hàng này được phép bắt đầu. Thời gian cần thiết để giao hàng cho khách hàng i được ký hiệu là d_i và $t_{i,j}$ biểu diễn thời gian di chuyển từ điểm i đến điểm j , với $i, j \in \{0, 1, \dots, n\}$ và $i \neq j$.

Một lời giải của bài toán được biểu diễn bởi một hoán vị

$$s[1], s[2], \dots, s[n]$$

của các khách hàng $1, 2, \dots, n$, xác định thứ tự nhân viên giao hàng ghé thăm các khách hàng. Thời điểm đến và thời điểm bắt đầu phục vụ tại từng khách hàng được tính dựa trên thời điểm xuất phát, thời gian di chuyển và thời gian phục vụ các khách hàng trước đó. Nếu nhân viên đến khách hàng i trước thời điểm e_i , nhân viên phải chờ đến thời điểm e_i mới được bắt đầu giao hàng. Ngược lại, nếu thời điểm bắt đầu phục vụ vượt quá l_i , lộ trình đó không thỏa mãn ràng buộc cửa sổ thời gian.

Mục tiêu của TSPTW là tìm một lộ trình giao hàng hợp lệ sao cho mỗi khách hàng được phục vụ đúng một lần, không vi phạm ràng buộc về cửa sổ thời gian và tổng thời gian di chuyển là nhỏ nhất, bao gồm cả thời gian quay lại kho hàng sau khi đã phục vụ đủ n khách hàng.

Do việc chứng minh tồn tại một lời giải thỏa mãn các ràng buộc trong TSPTW là một bài toán NP-khó, các phương pháp heuristic và metaheuristic thường được sử dụng để tìm lời giải có chất lượng tốt trong thời gian chấp nhận được, đặc biệt với các bộ dữ liệu có số lượng khách hàng lớn [5].

CHƯƠNG 3. CÁC PHƯƠNG PHÁP GIẢI CHÍNH XÁC

3.1 Phương pháp quay lui

Với TSPTW, thuật toán quay lui sẽ xây dựng lộ trình giao hàng theo từng vị trí của lộ trình [6]. Dựa theo Thuật toán 1, tại bước thứ k , thuật toán chọn một khách hàng chưa được phục vụ để đặt vào vị trí $s[k]$ của lộ trình. Sau mỗi lần chọn, thuật toán cập nhật thời điểm đến, kiểm tra ràng buộc của sổ thời gian và tiếp tục với vị trí tiếp theo nếu lộ trình hiện tại còn khả thi. Khi đã chọn đủ n khách hàng, thuật toán tính tổng thời gian di chuyển của lộ trình, bao gồm cả thời gian quay về kho hàng, và cập nhật lời giải tốt nhất nếu lộ trình hiện tại có chi phí nhỏ hơn.

Thuật toán 1: Bước thứ k của thủ tục quay lui kết hợp nhánh cận

Function Backtracking(k):

```
for  $i \leftarrow 1$  to  $n$  do
    if  $visited[i] = false$  then
        arrivals[ $k$ ]  $\leftarrow$  Thời gian đến điểm  $i$ 
        // Kiểm tra điều kiện của sổ thời gian
        if arrivals[ $k$ ] >  $l[i]$  then
            continue
        visited[ $i$ ]  $\leftarrow true$ 
        Cập nhật các cấu trúc dữ liệu cần thiết
        if  $k = n$  then
            Ghi nhận lời giải và cập nhật lời giải tốt nhất
            // Nhánh cận
        else if có khả năng thu được lời giải tốt hơn then
            Backtracking( $k + 1$ )
        visited[ $i$ ]  $\leftarrow false$ 
        Khôi phục các cấu trúc dữ liệu cần thiết
```

Để giảm số lượng nhánh cần duyệt, thuật toán sử dụng một cận dưới đơn giản dựa vào thời gian di chuyển nhỏ nhất giữa hai điểm bất kỳ. Gọi min_time là giá trị nhỏ nhất của $t_{i,j}$ với $i \neq j$. Khi lộ trình hiện tại đã chọn được k khách hàng, tức là còn $n - k$ khách hàng chưa được phục vụ, lời giải đang xây dựng sẽ cần thêm tối thiểu một lượng chi phí $potential_time$ với

$$potential_time \geq (n - k + 1) \times min_time. \quad (3.1)$$

Gọi $current_time$ là chi phí hiện tại của lời giải, lúc đó chi phí tối thiểu của lộ trình đang xây dựng là $current_time + potential_time$. Nếu giá trị này không nhỏ hơn $best_time$ thì nhánh hiện tại không thể tạo ra lời giải tốt hơn lời giải tốt nhất

đã biết và thuật toán sẽ ngừng xét đến nhánh này.

Quay lui kết hợp nhánh cận là một thuật toán dễ hiểu, dễ cài đặt và luôn tìm được lời giải tối ưu nếu duyệt toàn bộ không gian tìm kiếm. Các ràng buộc của số thời gian cũng được kiểm tra trong quá trình xây dựng lời giải, nhờ đó nhiều lộ trình không khả thi được bỏ qua. Tuy nhiên, số lượng hoán vị của n khách hàng là $n!$ nên thuật toán vẫn phải xét một số lượng rất lớn lộ trình trong trường hợp xấu nhất. Kỹ thuật nhánh cận chỉ giúp giảm thời gian chạy trong thực tế nhưng không làm thay đổi bản chất bùng nổ tổ hợp của bài toán. Thuật toán này có độ phức tạp thời gian trong trường hợp xấu nhất là $\mathcal{O}(n!)$ và độ phức tạp không gian là $\mathcal{O}(n)$.

3.2 Phương pháp quy hoạch động với mặt nạ bit

Thuật toán này cải thiện cách duyệt so với quay lui bằng cách gom các lộ trình có cùng tập khách hàng đã phục vụ và cùng điểm kết thúc vào một trạng thái. Thay vì xét riêng từng hoán vị, thuật toán lưu lời giải tốt nhất cho mỗi trạng thái trung gian để tránh tính toán lại nhiều lộ trình con giống nhau [7].

Gọi $S \subseteq \{1, 2, \dots, N\}$ là tập khách hàng đã được phục vụ và $i \in S$ là khách hàng cuối cùng của lộ trình hiện tại. Định nghĩa $f(S, i, x)$ là tổng thời gian di chuyển nhỏ nhất của một lộ trình xuất phát từ kho hàng, phục vụ đúng các khách hàng trong tập S và kết thúc tại khách hàng i ở thời điểm x .

Giá trị này có thể được tính một cách truy hồi như sau:

$$f(S, i, x) = \begin{cases} \min_{j \in S \setminus \{i\}} (f(S \setminus \{i\}, j, x') + t_{j,i}) & , \text{ nếu } x \leq l_i \text{ với } x' = \max(e_i, x' + d_j + t_{j,i}) \\ +\infty & , \text{ ngược lại.} \end{cases} \quad (3.2)$$

Điều kiện khởi tạo được xét khi nhân viên đi trực tiếp từ kho hàng đến một khách hàng i . Khi đó, với mọi $i \in \{1, 2, \dots, N\}$,

$$f(\{i\}, i, x) = \begin{cases} x, & \text{ nếu } x \leq l_i, \text{ với } x = \max(e_i, t_{0,i}) \\ +\infty, & \text{ ngược lại.} \end{cases} \quad (3.3)$$

Thời gian sớm nhất để quay trở lại kho hàng là:

$$\text{min_comeback_time} = \min_{j \in \{1, 2, \dots, N\}} (f(\{1, 2, \dots, N\}, j, x') + t_{j,0}). \quad (3.4)$$

Trong thuật toán này, tập hợp S được biểu diễn bằng một dãy bit *mask* độ dài n , thể hiện những khách hàng nào đã được phục vụ. Khách hàng j đã được phục vụ

Thuật toán 2: Thuật toán quy hoạch động với mặt nạ bit

Input: Số khách hàng n , cửa sổ thời gian e, l , thời gian phục vụ d và ma trận thời gian di chuyển t

Output: Lộ trình tối ưu s và tổng thời gian nhỏ nhất $best_time$

$masks \leftarrow 2^n$

Khởi tạo $best_total[mask][i] \leftarrow +\infty$ với mọi $mask, i$

Khởi tạo $best_arrivals[mask][i]$ và $previous_points[mask][i]$

// Khởi tạo mặt nạ bit

for $i \leftarrow 1$ **to** n **do**

$arrival \leftarrow \max(t[0][i], e[i])$

if $arrival \leq l[i]$ **then**

$mask \leftarrow 2^{i-1}$

$best_total[mask][i] \leftarrow t[0][i]$

$best_arrivals[mask][i] \leftarrow arrival$

$previous_points[mask][i] \leftarrow 0$

// Xét tất cả trạng thái mặt nạ bit có thể

for $mask \leftarrow 0$ **to** $masks - 1$ **do**

for $current \leftarrow 1$ **to** n **do**

if $best_total[mask][current] < +\infty$ **then**

$finish_time \leftarrow best_arrivals[mask][current] + d[current]$

for $point \leftarrow 1$ **to** n **do**

if $point \in mask$ **then**

continue

$arrival \leftarrow \max(finish_time + t[current][point], e[point])$

if $arrival > l[point]$ **then**

continue

$new_mask \leftarrow mask \cup \{point\}$

$total_time \leftarrow best_total[mask][current] + t[current][point]$

if $total_time < best_total[new_mask][point]$ **then**

$best_total[new_mask][point] \leftarrow total_time$

$best_arrivals[new_mask][point] \leftarrow arrival$

$previous_points[new_mask][point] \leftarrow current$

else if $total_time = best_total[new_mask][point]$ **and**

$arrival < best_arrivals[new_mask][point]$ **then**

$best_arrivals[new_mask][point] \leftarrow arrival$

$previous_points[new_mask][point] \leftarrow current$

// Xét thêm thời gian quay về kho

$full_mask \leftarrow masks - 1$

$best_time \leftarrow +\infty$

for $i \leftarrow 1$ **to** n **do**

$total_time \leftarrow best_total[full_mask][i] + t[i][0]$

if $total_time < best_time$ **then**

$best_time \leftarrow total_time$

$last_point \leftarrow i$

Truy vết từ $last_point$ bằng bảng $previous_points$ để khôi phục s

nếu bit thứ $j - 1$ của $mask$ bằng 1. Lộ trình hiện tại sẽ được biểu diễn dưới dạng trạng thái $(mask, i)$, trong đó $mask$ là mặt nạ bit của tập hợp các khách hàng đã được phục vụ và i là khách hàng mới nhất được phục vụ. Thuật toán 2 trình bày mã giả của phương pháp quy hoạch động kết hợp mặt nạ bit.

So với quay lui nhánh cận, quy hoạch động với mặt nạ bit có sự cải thiện rõ ràng vì mỗi trạng thái $(mask, i)$ chỉ cần được xử lý một lần. Cách tiếp cận này tránh việc xét lặp lại nhiều lộ trình con có cùng tập khách hàng đã phục vụ và cùng điểm kết thúc. Tuy nhiên, số lượng trạng thái vẫn tăng theo hàm mũ theo n , do có 2^n tập khách hàng khác nhau và tối đa n điểm kết thúc cho mỗi tập. Vì vậy, phương pháp này vẫn không thể thực thi hiệu quả trên các bộ dữ liệu lớn.

Độ phức tạp thời gian của thuật toán là $\mathcal{O}(2^n \times n^2)$ vì có tối đa $2^n \times n$ trạng thái và từ mỗi trạng thái thuật toán có thể thử mở rộng sang tối đa n khách hàng chưa phục vụ, còn độ phức tạp không gian là $\mathcal{O}(2^n \times n)$.

CHƯƠNG 4. MÔ HÌNH HÓA BÀI TOÁN

Chương này trình bày các cách mô hình hóa TSPTW nhằm phục vụ các mô hình giải chính xác của thư viện OR-Tools.

Ký hiệu V là tập các khách hàng, tức là $V = \{0, 1, 2, \dots, n\}$, ở đây kho hàng được coi là khách hàng thứ 0. Quy ước $a_0 = e_0 = l_0 = d_0 = 0$.

4.1 Mô hình quy hoạch nguyên tuyến tính cho bộ giải PywrapLP

Biến quyết định. Mô hình này gồm hai nhóm biến quyết định sau:

1. Các biến nhị phân $x_{i,j}$ ($i, j \in V$), định nghĩa như sau:

$$x_{i,j} = \begin{cases} 1, & \text{nếu nhân viên di chuyển trực tiếp từ điểm } i \text{ đến điểm } j, \\ 0, & \text{ngược lại.} \end{cases} \quad (4.1)$$

2. Biến thời gian a_i , biểu diễn thời điểm bắt đầu phục vụ tại điểm i ($a_i \in [e_i, l_i], \forall i \in V$).

Ràng buộc.

1. Mỗi khách hàng được phục vụ đúng một lần, tức là người giao hàng đi vào mỗi khách hàng đúng một lần và ra khỏi mỗi khách hàng đúng một lần:

$$\sum_{j \in V \setminus \{i\}} x_{i,j} = \sum_{j \in V \setminus \{i\}} x_{j,i} = 1, \forall i \in V. \quad (4.2)$$

2. Nếu người giao hàng đi từ khách hàng i đến khách hàng j ($x_{i,j} = 1, j > 0$) thì:

$$a_j \geq a_i + d_i + t_{i,j}. \quad (4.3)$$

Ràng buộc này sẽ được tuyến tính hóa thông qua một giá trị M rất lớn, tức là:

$$a_j \geq a_i + d_i + t_{i,j} - M \times (1 - x_{i,j}), \forall i, j \in V, j > 0. \quad (4.4)$$

Hàm mục tiêu. Mục tiêu của bài toán là tìm lộ trình có tổng thời gian di chuyển nhỏ nhất. Vì $x_{i,j} = 1$ nếu người giao hàng đi từ khách hàng i đến khách hàng j nên tổng thời gian di chuyển được biểu diễn bởi:

$$\min \sum_{i \in V} \sum_{j \in V \setminus \{i\}} x_{i,j} t_{i,j}. \quad (4.5)$$

4.2 Mô hình quy hoạch ràng buộc cho bộ giải CP-SAT

Biến quyết định. Mô hình sử dụng hai nhóm biến sau:

1. Nhóm biến $next_i \in V, \forall i \in V$, nhận giá trị j nếu người giao hàng di chuyển trực tiếp từ điểm i đến điểm j .
2. Nhóm biến a_i với định nghĩa như đã mô tả ở Phần 4.1.

Ràng buộc.

1. Các giá trị $next_i$ phải đôi một khác nhau:

$$\text{AllDifferent}(next_0, next_1, \dots, next_n). \quad (4.6)$$

2. Nếu người giao hàng đi từ khách hàng i đến khách hàng j ($next_i = j, j > 0$) thì:

$$a_j \geq a_i + d_i + t_{i,j}. \quad (4.7)$$

Hàm mục tiêu. Hàm mục tiêu của mô hình này là:

$$\min \sum_{i \in V} t_{i, next_i}. \quad (4.8)$$

Tuy nhiên, trong quá trình cài đặt mô hình, $next_i$ là một biến quyết định chứ không phải một chỉ số cố định, nên trong lập trình không thể viết:

$$\text{model.Minimize}(\text{sum}(t[i][next[i]])).$$

Vì thế, mô hình sử dụng biến phụ c_i để biểu diễn chi phí di chuyển từ điểm i đến điểm kế tiếp của nó:

$$c_i = t_{i, next_i}, \forall i \in V. \quad (4.9)$$

Khi đó, hàm mục tiêu được viết lại thành:

$$\min \sum_{i \in V} c_i. \quad (4.10)$$

CHƯƠNG 5. CÁC PHƯƠNG PHÁP THAM LAM

Chương này trình bày hai phương pháp tham lam được sử dụng để giải TSPTW.

Lựa chọn theo e_i . Phương pháp thứ nhất lựa chọn các khách hàng theo giá trị bắt đầu được phép phục vụ e_i nhỏ nhất. Tại mỗi bước, thuật toán xét các khách hàng chưa được phục vụ và chọn khách hàng có e_i nhỏ nhất mà vẫn thỏa mãn ràng buộc về cửa sổ thời gian. Lộ trình thu được từ phương pháp này có dạng $s[1], s[2], \dots, s[n]$ với

$$e_{s[i]} < e_{s[i+1]}, \forall i \in \{1, 2, \dots, n-1\}. \quad (5.1)$$

Lựa chọn theo l_i . Phương pháp thứ hai lựa chọn theo giá trị muộn nhất có thể bắt đầu phục vụ l_i nhỏ nhất. Phương pháp này gần giống với phương pháp thứ nhất, chỉ khác ở chỗ Phương trình 5.1 được thay bởi:

$$l_{s[i]} < l_{s[i+1]}, \forall i \in \{1, 2, \dots, n-1\}. \quad (5.2)$$

Tuy nhiên, ràng buộc về cửa sổ thời gian có thể khiến hai phương pháp trên không đưa ra được một lời giải chấp nhận được, đặc biệt là khi cửa sổ thời gian quá hẹp còn khoảng cách giữa hai khách hàng là quá lớn. Độ phức tạp thời gian và không gian của mỗi phương pháp lần lượt là $\mathcal{O}(n^2)$ và $\mathcal{O}(n)$.

CHƯƠNG 6. CÁC PHƯƠNG PHÁP TÌM KIẾM CỤC BỘ

Các phương pháp tìm kiếm cục bộ được sử dụng trong các bài toán tối ưu tổ hợp nhằm đưa ra lời giải đủ tốt trong một khoảng thời gian nhất định, thay vì phải duyệt hết toàn bộ các lời giải có thể của bài toán để tìm ra lời giải tối ưu. Tìm kiếm cục bộ bắt đầu bằng việc khởi tạo lời giải ban đầu, sau đó liên tục cải thiện bằng cách xét các lời giải lân cận của nó. Cách tiếp cận này không đảm bảo luôn tìm được lời giải tối ưu toàn cục, nhưng tìm kiếm cục bộ có tốc độ tính toán nhanh, cách cài đặt tương đối đơn giản và có thể áp dụng cho các bộ dữ liệu với kích thước lớn. Ngoài ra, tìm kiếm cục bộ cũng rất linh hoạt khi có thể thay đổi cách sinh lân cận, tiêu chí chấp nhận lời giải mới hoặc kết hợp với lời giải ban đầu từ phương pháp tham lam để thu được nghiệm tốt trong thời gian ngắn.

6.1 Tổng quan về các phương pháp tìm kiếm cục bộ được sử dụng

6.1.1 Khởi tạo lời giải ban đầu

Các thuật toán tìm kiếm cục bộ trong báo cáo này sử dụng những cách thức khởi tạo lời giải ban đầu gần giống với các phương pháp tham lam ở Chương 5, nhưng sẽ tạm bỏ qua điều kiện về cửa sổ thời gian. Các phương pháp tham lam ở Chương 5 sẽ dừng lại ngay khi không còn khách hàng nào thỏa mãn điều kiện về cửa sổ thời gian, trong khi việc tạm chấp nhận vi phạm ràng buộc về cửa sổ thời gian đảm bảo tạo ra lộ trình ban đầu gồm đủ n khách hàng, và độ vi phạm ràng buộc của lời giải sẽ được giảm thông qua các bước lặp của tìm kiếm cục bộ.

6.1.2 Toán tử biến đổi

Đổi chỗ hai khách hàng (Swap). Toán tử này chọn hai khách hàng i, j ($i \in \{1, 2, \dots, n-1\}, j \in \{i+1, i+2, \dots, n\}$) và đổi chỗ hai khách hàng này. Lời giải thu được sẽ có dạng:

$$s[1], s[2], \dots, s[i-1], s[j], s[i+1], \dots, s[j-1], s[i], s[i+1], \dots, s[n].$$

Chèn một khách hàng sang vị trí khác (Relocate). Toán tử này chọn hai khách hàng i, j ($i, j \in \{i+1, i+2, \dots, n\}, i \neq j$) và chuyển khách hàng i đến vị trí ngay sau khách hàng j . Lời giải thu được sẽ có dạng:

$$s[1], s[2], \dots, s[i-1], s[i+1], \dots, s[j-1], s[j], s[i], s[i+1], \dots, s[n].$$

Đảo ngược thứ tự phục vụ của một dãy khách hàng liên tiếp (2-opt move). Toán tử này chọn một dãy khách hàng, bắt đầu từ khách hàng i , kết thúc ở khách

hàng j ($i \in \{1, 2, \dots, n-2\}, j \in \{i+2, i+3, \dots, n\}$) và đảo ngược thứ tự phục vụ của dãy khách hàng này. Lời giải thu được sẽ có dạng:

$$s[1], s[2], \dots, s[i-1], s[j], s[j-1], \dots, s[i+1], s[i], s[j+1], \dots, s[n].$$

6.1.3 Đánh giá chất lượng lời giải

Báo cáo đánh giá chất lượng của một lời giải s dựa trên độ vi phạm ràng buộc đối với mỗi khách hàng. Với giá trị $a[i]$ được định nghĩa ở Phần 4.1, độ vi phạm ràng buộc ở khách hàng i được định nghĩa bởi:

$$violation[i] = \max(a[i] - l[i], 0). \quad (6.1)$$

Độ vi phạm ràng buộc của s là tổng độ vi phạm ràng buộc tại các khách hàng, tức là:

$$violation(s) = \sum_{i=1}^n violation[i]. \quad (6.2)$$

Tổng thời gian di chuyển của người giao hàng, tính cả thời gian quay về kho hàng, được tính bởi:

$$time(s) = \sum_{i=1}^n t_{s[i], s[i+1]}, \quad s[n+1] = s[1]. \quad (6.3)$$

Chất lượng của một lời giải s sẽ được tính như sau:

$$fitness(s) = time(s) + \beta \times violation(s), \quad (6.4)$$

trong đó β là một hằng số phạt lớn hơn 1, nhằm hướng việc tìm kiếm tới những lời giải có độ vi phạm ràng buộc thấp hơn.

6.2 Thuật toán tìm kiếm leo đồi

Thuật toán tìm kiếm leo đồi (Hill Climbing - HC) là một trong những thuật toán tìm kiếm cục bộ cơ bản nhất [8]. Sau khi khởi tạo lời giải ban đầu, với mỗi bước lặp, thuật toán sử dụng các toán tử được định nghĩa sẵn để di chuyển đến các lời giải láng giềng. Nếu thu được ít nhất một lời giải có chất lượng tốt hơn, lời giải cũ sẽ bị thay thế và vòng lặp tiếp theo sẽ bắt đầu từ lời giải mới. Thuật toán 3 trình bày cách hoạt động của thuật toán HC.

Thuật toán trong báo cáo sử dụng cơ chế cải thiện đầu tiên (First improvement). Nhằm tăng tính đa dạng của lời giải, ở mỗi vòng lặp, thuật toán sẽ hoán đổi ngẫu

Thuật toán 3: Thuật toán leo đồi

Input: Lời giải ban đầu s

Output: Lời giải tốt nhất tìm được s

do

$s' = \text{explore_neighbourhood}(s)$

if $\text{fitness}(s') < \text{fitness}(s)$ **then**

$s \leftarrow s'$

until *no more improvent or time limit reached;*

nhiên thứ tự sử dụng các toán tử biến đổi.

6.3 Thuật toán mô phỏng quá trình tối ử vật liệu

Tối ử vật liệu là quá trình đưa một chất rắn về trạng thái năng lượng thấp sau khi đã tăng nhiệt độ. Thuật toán tìm kiếm mô phỏng quá trình tối ử vật liệu (Simulated Annealing - SA) cũng xuất phát từ ý tưởng đó, thông qua việc kết hợp tính khám phá và tính khai thác [9].

Thuật toán 4: Thuật toán mô phỏng quá trình tối ử vật liệu

Input: Lời giải ban đầu s , nhiệt độ ban đầu T_0 , nhiệt độ cuối T_{min} và hệ số làm nguội α

Output: Lời giải tốt nhất tìm được s^*

$s^* \leftarrow s$

$T \leftarrow T_0$

while $T > T_{min}$ **and** *time limit not reached* **do**

$s' \leftarrow \text{random_neighbour}(s)$

$\Delta \leftarrow \text{fitness}(s') - \text{fitness}(s)$

if $\Delta < 0$ **or** $\text{random}(0, 1) < e^{-\Delta/T}$ **then**

$s \leftarrow s'$

if $\text{fitness}(s) < \text{fitness}(s^*)$ **then**

$s^* \leftarrow s$

$T \leftarrow \alpha \times T$

Thuật toán 4 trình bày cách hoạt động của SA cho TSPTW. Nếu lời giải láng giềng s' có chất lượng tốt hơn lời giải hiện tại s thì nó sẽ được chấp nhận. Ngược lại, nếu s' tệ hơn s , thuật toán vẫn có thể chấp nhận s' với xác suất

$$p = e^{\frac{\text{fitness}(s) - \text{fitness}(s')}{T}}. \quad (6.5)$$

Nhiệt độ T điều khiển mức độ sẵn sàng chấp nhận các lời giải tệ hơn. Khi T còn lớn, thuật toán có khả năng chấp nhận nhiều bước đi làm tăng chi phí, nhờ đó có thể thoát khỏi các lời giải tối ưu cục bộ. Khi T giảm dần, xác suất chấp nhận các

lời giải tệ hơn cũng giảm, và thuật toán chuyển dần sang trạng thái khai thác quanh các lời giải tốt đã tìm được. Khi T tiến gần về 0, SA hoạt động gần giống một phiên bản leo đồi ngẫu nhiên vì hầu như không còn chấp nhận các bước đi làm xấu lời giải.

Lịch làm nguội là quy tắc dùng để giảm nhiệt độ T qua các lần lặp. Báo cáo sử dụng lịch làm nguội theo cấp số nhân, trong đó sau mỗi bước lặp nhiệt độ được cập nhật bởi

$$T \leftarrow \alpha \times T, \quad 0 < \alpha < 1. \quad (6.6)$$

Giá trị α càng gần 1 thì nhiệt độ giảm càng chậm, thuật toán có nhiều thời gian khám phá không gian nghiệm hơn nhưng thời gian chạy cũng tăng lên. Ngược lại, nếu α nhỏ, thuật toán chạy nhanh hơn nhưng dễ hội tụ sớm vào một lời giải chưa đủ tốt.

CHƯƠNG 7. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

7.1 Cấu hình thực nghiệm

7.1.1 Bộ dữ liệu thử nghiệm

Bộ dữ liệu được sử dụng để thử nghiệm các thuật toán được xây dựng với mười kích thước khác nhau, trong đó kích thước của mẫu dữ liệu được tính bằng số khách hàng n . Mỗi mẫu dữ liệu được thiết kế sao cho tồn tại ít nhất một lời giải chấp nhận được. Để đảm bảo điều này, quá trình sinh dữ liệu bắt đầu bằng cách tạo trước một lộ trình khả thi, tạo ma trận thời gian di chuyển t và mảng d biểu diễn thời gian phục vụ tại mỗi khách hàng, từ đó tính lần lượt thời gian bắt đầu phục vụ a_i tại từng khách hàng trên lộ trình đó. Từ đó, cửa sổ thời gian của khách hàng i được sinh ra sao cho $e_i \leq a_i \leq l_i$. Bộ dữ liệu được sinh ra nhằm đảm bảo rằng nếu chỉ sử dụng các phương pháp tham lam mà không có sự cải thiện lời giải thì sẽ chỉ thu được lời giải không chấp nhận được trong phần lớn trường hợp.

Loại dữ liệu	Kích thước	PywrapLP		CP-SAT	
		Giá trị hàm mục tiêu	Thời gian thực thi (giây)	Giá trị hàm mục tiêu	Thời gian thực thi (giây)
Thông thường	5	244	0.026	244	0.166
	10	376	0.048	376	0.018
	20	433	33.413	433	1.164
	50	1115	601.858	660	606.258
	100	$+\infty$	$+\infty$	1646	606.542
	≥ 200	$+\infty$	$+\infty$	$+\infty$	$+\infty$
Phân cụm	5	524	0.033	524	0.036
	10	506	7.231	506	0.130
	20	642	599.949	638	2.400
	50	$+\infty$	$+\infty$	904	193.483
	100	$+\infty$	$+\infty$	1405	607.194
	≥ 200	$+\infty$	$+\infty$	$+\infty$	$+\infty$

Bảng 7.1: Kết quả thực nghiệm của các bộ giải chính xác. Thời gian thực thi được làm tròn đến ba chữ số sau dấu thập phân. Giá trị được in đậm là giá trị tối ưu nếu bộ giải tìm được lời giải tối ưu. Giá trị hàm mục tiêu và thời gian thực thi là $+\infty$ nếu bộ giải không tìm được một lời giải chấp nhận được trong thời gian cho phép.

Bộ dữ liệu gồm hai nhóm chính. Nhóm thứ nhất là các mẫu dữ liệu thông thường, trong đó thời gian di chuyển giữa các khách hàng được phân bố tương đối ngẫu nhiên trên mặt phẳng. Nhóm thứ hai là các mẫu dữ liệu phân cụm. Trong nhóm này, các khách hàng được chia thành hai cụm riêng biệt. Khoảng cách giữa hai khách hàng trong cùng một cụm sẽ nhỏ hơn nhiều so với khoảng cách giữa hai

Loại dữ liệu	Kích thước	Giá trị hàm mục tiêu			Thời gian thực thi (giây)	Số vòng lặp
		Nhỏ nhất	Lớn nhất	Trung bình		
Thông thường	5	244	244	244 ± 0	0.001	9
	10	376	419	394 ± 18	0.006	16
	20	434	455	448 ± 10	0.100	51
	50	869	956	899 ± 40	2.668	419
	100	1645	1673	1657 ± 12	103.757	326
	200	3001	3106	3069 ± 48	1592.638	700
	300	5311	5725	5471 ± 182	2424.469	775
	500	11538	11845	11672 ± 128	2511.176	948
	750	19858	20315	20099 ± 187	2473.479	1155
	1000	29947	32788	31629 ± 1218	2405.219	1073
Phân cụm	5	524	524	524 ± 0	0.001	2
	10	506	512	509 ± 2	0.013	20
	20	647	660	657 ± 7	0.105	59
	50	989	1004	1054 ± 81	2.543	207
	100	1802	1807	1814 ± 14	86.615	481
	200	3494	3507	3511 ± 15	1512.533	1041
	300	5167	5304	5324 ± 138	2424.791	1369
	500	9752	11115	10325 ± 577	2501.324	1905
	750	16406	17360	16846 ± 393	2418.599	1995
	1000	33007	37412	35524 ± 1853	2403.017	1296

Bảng 7.2: Kết quả thực nghiệm trên ba lần chạy độc lập của thuật toán tìm kiếm leo đồi. Giá trị trung bình và độ lệch chuẩn của các giá trị hàm mục tiêu được làm tròn đến số nguyên gần nhất. Số vòng lặp là tổng số vòng lặp của hai cách khởi tạo lời giải.

khách hàng thuộc hai cụm khác nhau. Điều này làm cho việc tìm ra lời giải tối ưu trở nên khó hơn với các cách khởi tạo ban đầu đơn giản, vì một lộ trình ban đầu có thể di chuyển qua lại giữa hai cụm nhiều lần và tạo tổng thời gian di chuyển lớn.

7.1.2 Cài đặt tham số

Với mỗi mẫu dữ liệu, thuật toán HC sẽ bắt đầu lần lượt với hai chiến lược khởi tạo lời giải ở Mục 6.1.1. Với mỗi chiến lược, thuật toán sẽ được thực thi trong hai mươi phút. Kết quả thu được là kết quả tốt hơn trong hai chiến lược khởi tạo lời giải ban đầu. Hệ số phạt β của thuật toán là 1000000.

Với thuật toán SA, nhiệt độ ban đầu $T_0 = 10000000$, nhiệt độ cuối cùng $T_{min} = 1$, hệ số làm nguội $\alpha = 0.99995$ và hệ số phạt $\beta = 10000$.

7.2 Kết quả của các bộ giải chính xác thuộc thư viện OR-Tools

Bảng 7.1 trình bày giá trị hàm mục tiêu tốt nhất tìm được và thời gian thực thi của hai bộ giải PywrapLP và CP-SAT, trong đó mỗi bộ giải được phép chạy trong

Loại dữ liệu	Kích thước	Giá trị hàm mục tiêu			Thời gian thực thi (giây)
		Nhỏ nhất	Lớn nhất	Trung bình	
Thông thường	5	244	244	244 ± 0	42.281
	10	376	376	376 ± 0	44.262
	20	434	462	452 ± 13	56.843
	50	811	852	833 ± 17	64.790
	100	1715	1751	1735 ± 15	112.173
	200	3502	3613	3543 ± 50	182.511
	300	5441	5520	5488 ± 34	264.690
	500	9686	9739	9713 ± 22	415.991
	750	15344	15449	15414 ± 49	612.835
	1000	21150	21256	21219 ± 49	887.286
Phân cụm	5	524	524	524 ± 0	46.651
	10	506	506	506 ± 0	43.517
	20	638	642	640 ± 2	58.582
	50	957	974	968 ± 8	76.996
	100	1629	1679	1652 ± 21	110.406
	200	3059	3311	3149 ± 115	188.006
	300	4514	4713	4645 ± 93	267.082
	500	8100	8561	8329 ± 188	412.276
	750	13025	13421	13166 ± 181	594.308
	1000	18365	18978	18698 ± 253	852.312

Bảng 7.3: Kết quả thực nghiệm trên ba lần chạy độc lập của thuật toán tìm kiếm mô phỏng quá trình tối ưu vật liệu. Giá trị trung bình và độ lệch chuẩn của các giá trị hàm mục tiêu được làm tròn đến số nguyên gần nhất.

mười phút. Đối với các bộ dữ liệu có kích thước nhỏ, cả hai bộ giải đều cho ra lời giải tối ưu. Tuy nhiên, khi kích thước dữ liệu ngày càng tăng thì hiệu năng của hai bộ giải cũng giảm đi, và cả hai đều không tìm được bất kỳ lời giải chấp nhận được nào khi kích thước mẫu dữ liệu là 200. Do đó, các phương pháp tìm kiếm cục bộ là cần thiết để tìm ra lời giải đủ tốt cho các mẫu dữ liệu có kích thước lớn.

7.3 Kết quả của các phương pháp tìm kiếm cục bộ

7.3.1 Thuật toán tìm kiếm leo đồi

Bảng 7.2 trình bày kết quả thực nghiệm của thuật toán HC trên hai nhóm dữ liệu thông thường và phân cụm. Nhìn chung, kết quả giữa ba lần chạy độc lập không có sự chênh lệch quá lớn, thể hiện qua độ lệch chuẩn tương đối nhỏ so với giá trị trung bình ở hầu hết các kích thước dữ liệu. So với việc chỉ sử dụng các thuật toán tham lam thông thường để xây dựng lời giải ban đầu, HC cho kết quả tốt hơn nhờ khả năng tiếp tục cải thiện lộ trình sau bước khởi tạo. Hiệu quả này đặc biệt có ý nghĩa

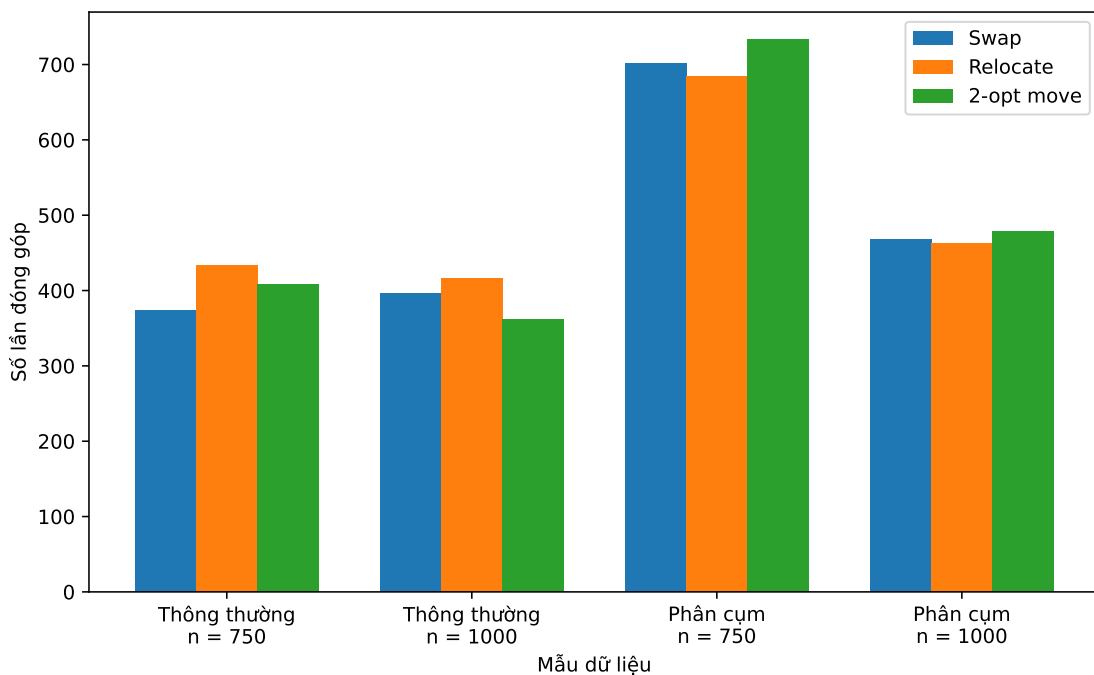
với các mẫu dữ liệu có kích thước lớn hoặc dữ liệu phân cụm, khi lời giải tham lam ban đầu thường còn cách khá xa lời giải tốt.

7.3.2 Thuật toán tìm kiếm mô phỏng quá trình tôi ủ vật liệu

Bảng 7.3 trình bày kết quả thực nghiệm của thuật toán SA trên hai nhóm dữ liệu thông thường và phân cụm. Trong các lần chạy, thuật toán sử dụng chung các tham số T_0 , T_{min} và α , do đó số lần lặp của mỗi lần chạy là như nhau đối với cùng một cấu hình dữ liệu. Kết quả thu được trên ba lần chạy độc lập cũng không có nhiều biến động, thể hiện qua độ lệch chuẩn tương đối nhỏ so với giá trị trung bình ở phần lớn các kích thước dữ liệu.

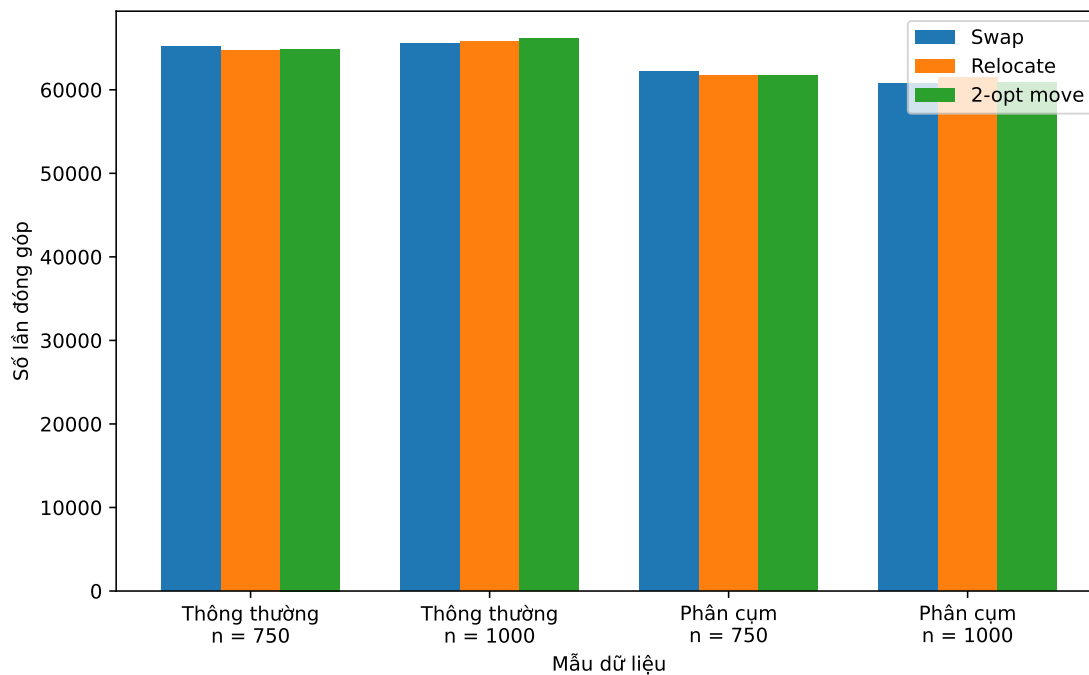
So với thuật toán HC, thuật toán SA cho thấy sự cải thiện rõ rệt về chất lượng lời giải ở nhiều mẫu dữ liệu, đồng thời thời gian chạy cũng thấp hơn. Nguyên nhân là SA không chỉ chấp nhận các lời giải láng giềng tốt hơn mà còn có thể chấp nhận một số lời giải tệ hơn với xác suất phụ thuộc vào nhiệt độ T . Cơ chế này giúp thuật toán có khả năng thoát khỏi các lời giải tối ưu cục bộ, tiếp tục khám phá các vùng nghiệm khác và tìm được lời giải tốt hơn trong thời gian thực thi ngắn hơn.

7.4 Phân tích mức độ đóng góp của các toán tử



Hình 7.1: Số lần cải thiện lời giải của từng toán tử trong thuật toán tìm kiếm leo đồi.

Hình 7.1 và Hình 7.2 trình bày mức độ đóng góp của các toán tử trong từng thuật toán tìm kiếm cục bộ. Cả hai hình đều phân tích lần chạy có kết quả tốt nhất trong các lần chạy được trình bày ở Bảng 7.2 và Bảng 7.3, xét trên hai mẫu dữ liệu có kích thước lớn nhất của từng nhóm dữ liệu.



Hình 7.2: Số lần cập nhật lời giải của từng toán tử trong thuật toán tìm kiếm mô phỏng quá trình tôi ủ vật liệu, bao gồm các lần cải thiện lời giải và các lần chấp nhận lời giải tệ hơn để thoát khỏi điểm tối ưu cục bộ.

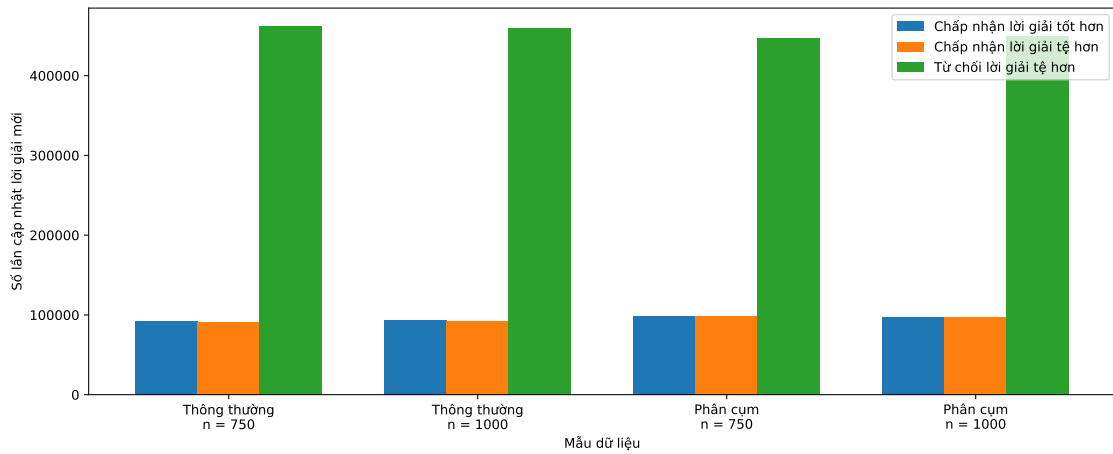
Các hình cho thấy cả ba toán tử swap, relocate và 2-opt move đều có đóng góp tương đương trong quá trình tiến tới lời giải cuối cùng. Điều này cho thấy mỗi toán tử đảm nhiệm một vai trò khác nhau trong quá trình tìm kiếm: Swap tạo ra các điều chỉnh nhỏ, relocate thay đổi vị trí của khách hàng, còn 2-opt move tái cấu trúc các đoạn lộ trình dài hơn.

So với Hình 7.1, số lần đóng góp của các toán tử trong Hình 7.2 vượt trội hơn đáng kể. Nguyên nhân là SA có khả năng chấp nhận cả những lời giải tệ hơn trong một số trường hợp, nhờ đó tiếp tục khám phá thêm nhiều vùng nghiệm thay vì dừng sớm tại một lời giải tối ưu cục bộ. Vì vậy, thuật toán SA thường thu được lời giải tốt hơn so với thuật toán HC trên các mẫu dữ liệu lớn.

7.5 Thống kê về các lần cập nhật lời giải của thuật toán tìm kiếm mô phỏng quá trình tôi ủ vật liệu

Hình 7.3 trình bày thống kê chi tiết về các lần cập nhật lời giải của thuật toán SA. Các lần cập nhật ở đây bao gồm cả trường hợp lời giải láng giềng tốt hơn lời giải hiện tại và trường hợp thuật toán chấp nhận một lời giải tệ hơn theo xác suất phụ thuộc vào nhiệt độ T . Kết quả cho thấy tỷ lệ cập nhật lời giải xấp xỉ $1/4$ tổng số lần xét lời giải láng giềng và tỷ lệ chấp nhận lời giải tệ hơn là khoảng $1/8$, nghĩa là thuật toán không bị giới hạn trong việc chỉ nhận các bước cải thiện trực tiếp.

Tỷ lệ cập nhật này cho thấy cách cài đặt tham số ở phần cấu hình thực nghiệm,



Hình 7.3: Thống kê về các lần cập nhật lời giải của từng toán tử trong thuật toán tìm kiếm mô phỏng quá trình tôi ủ vật liệu.

bao gồm T_{init} , T_{min} và α , giúp thuật toán duy trì khả năng khám phá không gian nghiệm trong quá trình tìm kiếm. Nhờ vẫn chấp nhận một số lời giải tệ hơn, SA có thêm cơ hội thoát khỏi các lời giải tối ưu cục bộ và tiếp tục di chuyển tới những vùng nghiệm có chất lượng tốt hơn. Điều này góp phần giải thích vì sao SA thu được kết quả tốt hơn so với thuật toán leo đồi trong nhiều mẫu dữ liệu lớn.

CHƯƠNG 8. KẾT LUẬN

Báo cáo đã trình bày quá trình tìm hiểu và giải quyết bài toán TSPTW thông qua nhiều hướng tiếp cận khác nhau. Đầu tiên, báo cáo trình bày các phương pháp giải chính xác như quay lui kết hợp nhánh cận và quy hoạch động với mật nạ bit. Đây là các phương pháp cơ bản và dễ cài đặt, nhưng lại có độ phức tạp thời gian tính theo hàm mũ và chỉ có thể áp dụng cho các mẫu dữ liệu nhỏ. Tiếp theo, báo cáo trình bày cách mô hình hóa bài toán để đưa vào các bộ giải chính xác thuộc thư viện OR-Tools, tạo ra mốc so sánh cơ sở cho các thuật toán tìm kiếm cục bộ. Tuy nhiên, kết quả thực nghiệm cho thấy khi số lượng khách hàng tăng lên, thời gian chạy của các bộ giải chính xác tăng rất nhanh và nhiều trường hợp không tìm được lời giải chấp nhận được trong thời gian cho phép. Bên cạnh đó, báo cáo đã xây dựng và đánh giá các phương pháp tìm kiếm cục bộ, bao gồm thuật toán leo đồi và thuật toán mô phỏng quá trình tôi ủ vật liệu. Các thuật toán này bắt đầu từ lời giải ban đầu và liên tục cải thiện lời giải thông qua các toán tử biến đổi như swap, relocate và 2-opt move. Kết quả thực nghiệm cho thấy các phương pháp tìm kiếm cục bộ có khả năng tìm được lời giải đủ tốt trong thời gian hợp lý, đặc biệt với những mẫu dữ liệu có kích thước lớn mà các bộ giải chính xác không còn hiệu quả.

Các phân tích thực nghiệm cũng được thực hiện nhằm làm rõ đặc điểm của từng phương pháp. Báo cáo đã so sánh kết quả giữa các bộ giải chính xác và các thuật toán tìm kiếm cục bộ, đánh giá sự ổn định giữa nhiều lần chạy độc lập, phân tích mức độ đóng góp của từng toán tử trong quá trình cải thiện lời giải và thống kê chi tiết các bước lặp của thuật toán SA. Những phân tích này giúp giải thích vì sao các thuật toán tìm kiếm cục bộ, đặc biệt là SA, có thể cho kết quả tốt hơn trong nhiều trường hợp nhờ khả năng thoát khỏi các điểm tối ưu cục bộ.

Trong tương lai, báo cáo có thể tiếp tục mở rộng hướng nghiên cứu bằng cách thử nghiệm các thuật toán phức tạp hơn như Tabu Search hoặc Large Neighbourhood Search, nhằm cải thiện quá trình tìm kiếm lời giải cho TSPTW.

TÀI LIỆU THAM KHẢO

- [1] Y. Azar **and** A. Vardi, “TSP with time windows and service time,” *arXiv preprint arXiv:1501.06158*, 2015.
- [2] F Farizal, A. Rachman **and** Y. W. Krisnantio, “Time Windows routing optimization for logistic service provider industry,” *PROCEEDING OF INTERNATIONAL SUMMIT ON EDUCATION, TECHNOLOGY, AND HUMANITY 2021*, **jourvol** 2727, **number** 1, **page** 030 012, 2023.
- [3] K. Fagerholt **and** M. Christiansen, “A travelling salesman problem with allocation, time window and precedence constraints—an application to ship scheduling,” *International Transactions in Operational Research*, **jourvol** 7, **number** 3, **pages** 231–244, 2000.
- [4] A. Bretin, G. Desaulniers **and** L.-M. Rousseau, “The traveling salesman problem with time windows in postal services,” *Journal of the Operational Research Society*, **jourvol** 72, **number** 2, **pages** 383–397, 2021.
- [5] M. W. Savelsbergh, “Local search in routing problems with time windows,” *Annals of Operations research*, **jourvol** 4, **number** 1, **pages** 285–305, 1985.
- [6] G. Kondrak **and** P. Van Beek, “A theoretical evaluation of selected backtracking algorithms,” *Artificial Intelligence*, **jourvol** 89, **number** 1-2, **pages** 365–387, 1997.
- [7] V. Shoran, H. Dabas, G. Rathee, N. Rakesh, P. Agrawal **and** M. Gulhane, “Revolutionizing transportation and logistics: Dynamic programming and bit masking approach for optimizing the travelling salesman problem,” 2024.
- [8] M. A. Al-Betar, “ β -hill climbing: an exploratory local search,” *Neural Computing and Applications*, **jourvol** 28, **number** Suppl 1, **pages** 153–168, 2017.
- [9] S. Kirkpatrick, C. D. Gelatt Jr **and** M. P. Vecchi, “Optimization by simulated annealing,” *science*, **jourvol** 220, **number** 4598, **pages** 671–680, 1983.